

Ficheros

July 6, 2026

PAO25_25_05_Python_Files

July 5, 2026

PAO26-26 - Ficheros

Kevin Xavier Suasnavas Villarreal

[Link a mi GitHub](#)

1 1 Acceso a ficheros

1.1 1.1 Ficheros

Los **ficheros** (*files*) son objetos predefinidos (*built-in*) de Python que permiten almacenar información de forma permanente y acceder a ella desde un programa.

Algunas de sus características más importantes son:

- Los ficheros son objetos incorporados en Python (*built-in*).
- Permiten leer y escribir información almacenada en el disco.
- No pertenecen a los tipos de datos numéricos, secuencias o diccionarios, por lo que su manipulación se realiza mediante métodos específicos.
- Para trabajar con un fichero primero es necesario abrirlo mediante la función **open()**.

1.1.1 Abrir un fichero

La función **open()** permite crear un objeto de tipo fichero para poder leer o escribir información.

Su sintaxis general es la siguiente:

```
archivo = open(nombre_archivo, modo)
```

Donde:

- **nombre_archivo** corresponde al nombre o la ruta del fichero.
- **modo** indica la forma en que se abrirá el archivo (lectura, escritura, agregar información, etc.).

Si no se especifica el modo, Python abre el archivo en **modo lectura (r)** de forma predeterminada.

1.1.2 Consideraciones importantes

Al trabajar con archivos en Python es importante tener en cuenta los siguientes aspectos:

- Toda la información leída desde un fichero se obtiene inicialmente como una cadena de texto (**str**).
- Del mismo modo, al escribir datos en un fichero, Python almacena texto.
- Una vez finalizado el trabajo con el archivo, es recomendable cerrarlo utilizando el método **close()**, ya que esto libera los recursos utilizados por el sistema operativo y evita posibles errores.

1.2 Lectura de archivos

Una vez que un fichero ha sido abierto mediante la función **open()**, Python ofrece diferentes métodos para leer su contenido. Dependiendo de la necesidad, es posible leer todo el archivo de una sola vez, recorrerlo línea por línea o almacenar su contenido dentro de una lista.

1.2.1 Lectura completa de un archivo con **read()**

El método **read()** permite obtener todo el contenido del fichero como una única cadena de texto. Es recomendable utilizar este método cuando el archivo es pequeño y se desea trabajar con todo su contenido al mismo tiempo.

```
[1]: # =====  
# Lectura completa de un archivo con read()  
# =====  
  
# Abrimos el archivo en modo lectura.  
mi_fichero = open("res/multiple_lines.txt")  
  
# Leemos todo el contenido del archivo.  
contenido = mi_fichero.read()  
  
# Mostramos el contenido por pantalla.  
print(contenido)  
  
# Cerramos el archivo para liberar recursos.  
mi_fichero.close()
```

Este fichero
contiene tres
líneas de texto.

1.2.2 Lectura línea por línea utilizando un ciclo **for**

Los objetos de tipo fichero son iterables, por lo que pueden recorrerse mediante un ciclo **for**. Este método es muy útil cuando se trabaja con archivos grandes, ya que evita cargar todo el contenido en memoria.

```
[2]: # =====
# Lectura línea por línea
# =====

# Abrimos el archivo.
mi_fichero = open("res/multiple_lines.txt")

# Recorremos cada línea del fichero.
for linea in mi_fichero:

    # Mostramos la línea leída.
    print(linea)

# Cerramos el archivo.
mi_fichero.close()
```

Este fichero

contiene tres

líneas de texto.

1.2.3 Leer todas las líneas mediante `readlines()`

El método `readlines()` devuelve una lista donde cada elemento corresponde a una línea del archivo. Esto facilita recorrer posteriormente el contenido utilizando listas de Python.

```
[3]: # =====
# Leer un archivo como una lista
# =====

# Abrimos el archivo.
mi_fichero = open("res/multiple_lines.txt")

# Leemos todas las líneas y las almacenamos
# dentro de una lista.
lineas = mi_fichero.readlines()

# Cerramos el archivo.
mi_fichero.close()

# Mostramos la lista obtenida.
print(lineas)
```

```
['Este fichero\n', 'contiene tres\n', 'líneas de texto.\n']
```

1.2.4 Construcción de rutas independientes del sistema operativo

Los sistemas operativos utilizan separadores de rutas diferentes (/ o \). Para evitar problemas de compatibilidad, Python proporciona el módulo `os`, cuyo método `os.path.join()` construye rutas válidas para cualquier plataforma.

```
[4]: # =====  
# Construcción de rutas con os.path.join()  
# =====  
  
# Importamos el módulo os.  
import os  
  
# Construimos la ruta del archivo.  
ruta = os.path.join("res", "multiple_lines.txt")  
  
# Mostramos la ruta generada.  
print(ruta)
```

res\multiple_lines.txt

También es posible construir una ruta utilizando una lista de directorios y desempaquetándola con el operador `*`.

```
[5]: # =====  
# Construcción de rutas usando una lista  
# =====  
  
# Lista con los componentes de la ruta.  
dataset_file_path = [  
    "res",  
    "multiple_lines.txt"  
]  
  
# Construimos la ruta desempaquetando la lista.  
ruta = os.path.join(*dataset_file_path)  
  
# Mostramos el resultado.  
print(ruta)
```

res\multiple_lines.txt

1.2.5 Uso de la sentencia with

La forma más recomendada de trabajar con archivos en Python consiste en utilizar la sentencia `with`.

Esta estructura crea un **context manager**, el cual garantiza que el archivo será cerrado automáticamente al finalizar el bloque, incluso si ocurre algún error durante la ejecución.

```
[6]: # =====
# Lectura utilizando with
# =====

# Abrimos el archivo utilizando un context manager.
with open(ruta) as mi_fichero:

    # Recorremos el archivo línea por línea.
    for linea in mi_fichero:

        # end="" evita agregar una línea en blanco extra.
        print(linea, end="")
```

Este fichero
contiene tres
líneas de texto.

1.2.6 Abrir varios archivos simultáneamente

La sentencia **with** también permite abrir varios archivos dentro del mismo bloque de código, facilitando su lectura y asegurando que todos sean cerrados automáticamente al finalizar.

```
[7]: # =====
# Abrir varios archivos con with
# =====

# Construimos las rutas de ambos archivos.
ruta1 = os.path.join("res", "one_line.txt")
ruta2 = os.path.join("res", "multiple_lines.txt")

# Abrimos ambos archivos simultáneamente.
with open(ruta1) as fichero1, open(ruta2) as fichero2:

    # Mostramos el contenido del primer archivo.
    print(fichero1.readlines())

    print()

    # Mostramos el contenido del segundo archivo.
    print(fichero2.readlines())
```

```
['Hola mundo desde un fichero aqui estamos en machine.']
```

```
['Este fichero\n', 'contiene tres\n', 'lineas de texto.\n']
```

2 1.2 Modos de acceso

Al abrir un archivo mediante la función `open()`, es posible indicar el modo en el que se accederá al fichero. El modo de apertura determina si el archivo será utilizado para lectura, escritura o ambas operaciones.

Los modos más utilizados se muestran en la siguiente tabla.

Modo	Descripción
<code>r</code>	Solo lectura. Es el modo predeterminado.
<code>w</code>	Solo escritura. Si el archivo existe, su contenido se elimina antes de escribir.
<code>x</code>	Crea un nuevo archivo. Produce un error si el archivo ya existe.
<code>a</code>	Abre el archivo para agregar información al final. Si no existe, lo crea.
<code>r+</code>	Permite leer y escribir sobre el mismo archivo.
<code>b</code>	Acceso en modo binario. Se combina con otros modos (<code>rb</code> , <code>wb</code> , etc.).
<code>t</code>	Acceso en modo texto. Es el modo utilizado por defecto.

3 1.3 Acceso para escritura

Además de leer información, Python permite crear y modificar archivos utilizando distintos modos de escritura.

En el siguiente ejemplo se generará un archivo de texto y posteriormente se escribirán varias líneas utilizando los métodos `write()` y `writelines()`.

```
[8]: # =====  
# Crear un archivo y escribir información  
# =====  
  
# Importamos el módulo para trabajar con rutas.  
import os  
  
# Función que genera una lista de cadenas.  
def crear_lista(tamanyo):  
  
    # Creamos una lista vacía.  
    lista = []  
  
    # Agregamos los números convertidos a texto.  
    for i in range(tamanyo):  
  
        # Cada elemento termina con un salto de línea.  
        lista.append(str(i) + "\n")  
  
    # Devolvemos la lista creada.
```

```

    return lista

# Construimos la ruta del archivo.
ruta = os.path.join("res", "a_dummy.txt")

# Abrimos el archivo en modo escritura.
with open(ruta, "wt") as fichero:

    # Escribimos una línea de encabezado.
    fichero.write("Cabecera del ciclo del for en el ejemplo\n")

    # Generamos la lista.
    lista = crear_lista(10)

    # Escribimos todas las líneas de la lista.
    fichero.writelines(lista)

# Confirmamos que el archivo fue creado.
print("Archivo creado correctamente.")

```

Archivo creado correctamente.

3.1 Buffering

Cuando un programa escribe información en un archivo, los datos **no siempre se almacenan inmediatamente en el disco**.

Python utiliza un **buffer**, que es una zona de memoria temporal donde se guardan los datos antes de escribirlos definitivamente en el archivo.

El contenido del buffer se guarda en el disco cuando ocurre alguna de las siguientes acciones:

- Se cierra el archivo mediante **close()**.
- Se utiliza la sentencia **with**, que lo cierra automáticamente.
- Se invoca el método **flush()**, que fuerza la escritura inmediata del contenido del buffer.

```

[9]: # =====
# Ejemplo del uso del buffer
# =====

# Construimos la ruta del archivo.
ruta = os.path.join("res", "FicheroParaEscritura.txt")

# Abrimos el archivo para escritura.
fichero_write = open(ruta, "w")

# Escribimos una línea.
fichero_write.write("footer")

```

```

# Abrimos el mismo archivo para lectura.
fichero_read = open(ruta, "r")

print("Contenido antes de flush():")
print(fichero_read.readlines())

# Forzamos la escritura del buffer al disco.
fichero_write.flush()

# Regresamos al inicio del archivo para volver a leerlo.
fichero_read.seek(0)

print()

print("Contenido después de flush():")
print(fichero_read.readlines())

# Cerramos ambos archivos.
fichero_write.close()
fichero_read.close()

```

Contenido antes de flush():

```
[]
```

Contenido después de flush():

```
['footer']
```

3.1.1 Explicación del ejemplo

En este ejemplo se observa el funcionamiento del **buffer**.

1. Se abre un archivo para escritura y se almacena el texto "footer".
2. Antes de ejecutar **flush()**, la información puede seguir permaneciendo únicamente en el buffer.
3. Al llamar a **flush()**, el contenido se escribe físicamente en el archivo.
4. Finalmente, el archivo se vuelve a leer para comprobar que la información ya está disponible.

3.1.2 Conclusión

Al trabajar con archivos es recomendable utilizar la sentencia **with**, ya que administra automáticamente la apertura y el cierre de los ficheros, evitando pérdidas de información y liberando correctamente los recursos del sistema.

Cuando se utilizan archivos abiertos manualmente mediante **open()**, es importante recordar cerrar el archivo con **close()** o utilizar **flush()** cuando se necesite escribir inmediatamente el contenido almacenado en el buffer.

4 1.4 Archivos CSV

Los archivos **CSV** (*Comma-Separated Values*) son uno de los formatos más utilizados para almacenar y compartir datos tabulares.

Cada línea del archivo representa un registro y los campos se encuentran separados por un delimitador, que normalmente es una **coma** (,), aunque también pueden utilizarse otros caracteres como punto y coma (;) o barra vertical (|).

Este formato es ampliamente utilizado en áreas como **Ciencia de Datos**, **Machine Learning**, **Inteligencia Artificial** y **Análisis de Datos**, debido a su simplicidad y compatibilidad con numerosas herramientas.

4.0.1 Lectura de un archivo CSV

Python incorpora el módulo **csv**, el cual facilita la lectura de archivos CSV sin necesidad de procesar manualmente cada línea del fichero.

En el siguiente ejemplo se leerá un archivo denominado **tabla_operaciones.csv** y se mostrarán sus columnas por pantalla.

```
[10]: # =====  
# Lectura de un archivo CSV  
# =====  
  
# Importamos los módulos necesarios.  
import os  
import csv  
  
# Construimos la ruta del archivo.  
ruta = os.path.join("res", "tabla_operaciones.csv")  
  
# Abrimos el archivo utilizando un context manager.  
with open(ruta) as fichero:  
  
    # Creamos un lector CSV indicando  
    # que los datos están separados por comas.  
    data_reader = csv.reader(fichero, delimiter=",")  
  
    # Recorremos cada fila del archivo.  
    for linea in data_reader:  
  
        # Mostramos las dos primeras columnas.  
        print(linea[0] + " ---- " + linea[1])
```

```
Operacion ---- Descripcion  
a + b ---- suma a y b  
a - b ---- resta a menos b  
a / b ---- a dividido  
a // b ---- a dividido entre b (quitando decimales)
```

a % b ---- devuelve el resto de la división a/b (modulus)
a * b ---- a multiplicado por b
a ** b ---- a elevado a b

4.0.2 Escritura de un archivo CSV

Además de leer archivos CSV, Python también permite generar nuevos archivos utilizando el módulo `csv`.

En el siguiente ejemplo se leerá un archivo CSV separado por comas y posteriormente se generará una copia utilizando como separador el carácter `|`.

```
[11]: # =====  
# Escritura de un archivo CSV  
# =====  
  
# Construimos la ruta del archivo original.  
ruta = os.path.join("res", "tabla_operaciones.csv")  
  
# Abrimos el archivo para lectura.  
f = open(ruta)  
  
# Creamos el lector CSV.  
data_reader = csv.reader(f, delimiter=",")  
  
# Ruta del nuevo archivo.  
ruta_o = os.path.join("res", "4_tabla_operaciones.csv")  
  
# Abrimos el nuevo archivo para escritura.  
with open(ruta_o, "w", newline="") as f2:  
  
    # Creamos el escritor CSV utilizando  
    # el carácter "|" como separador.  
    csv_writer = csv.writer(f2, delimiter="|")  
  
    # Recorremos todas las filas.  
    for linea in data_reader:  
  
        # Mostramos la fila por pantalla.  
        print(linea)  
  
        # Escribimos la fila en el nuevo archivo.  
        csv_writer.writerow(linea)  
  
# Cerramos el archivo original.  
f.close()
```

```
['Operacion', 'Descripcion']  
['a + b', 'suma a y b']
```

```
['a - b', 'resta a menos b']
['a / b', 'a dividido']
['a // b', 'a dividido entre b (quitando decimales)']
['a % b', 'devuelve el resto de la división a/b (modulus)']
['a * b', 'a multiplicado por b']
['a ** b', 'a elevado a b']
```

4.0.3 Archivo generado

Después de ejecutar el programa se creará un nuevo archivo llamado `4_tabla_operaciones.csv` dentro de la carpeta **res**.

Su contenido tendrá un formato similar al siguiente:

```
Operacion|Descripcion
a + b|suma a y b
a - b|resta a menos b
a / b|a dividido
a // b|a dividido entre b (quitando decimales)
a % b|devuelve el resto de la división a/b (modulus)
a * b|a multiplicado por b
a ** b|a elevado a b
```

4.0.4 Conclusión

El módulo **csv** simplifica el procesamiento de archivos tabulares en Python. Gracias a las clases **csv.reader()** y **csv.writer()**, es posible leer, modificar y generar archivos CSV de forma sencilla, manteniendo la compatibilidad con herramientas ampliamente utilizadas como Microsoft Excel, LibreOffice Calc y diversas bibliotecas de análisis de datos como **Pandas**.

5 2 Docstrings

Los **docstrings** (*documentation strings*) son cadenas de texto utilizadas para documentar módulos, funciones, clases y otros objetos en Python.

Su objetivo es describir el funcionamiento del código para facilitar su comprensión, mantenimiento y reutilización.

Python almacena automáticamente esta documentación en el atributo especial **__doc__**, permitiendo consultarla durante la ejecución del programa.

5.0.1 Consultar la documentación de un objeto

Python incorpora la función **help()**, la cual permite visualizar la documentación asociada a un objeto, incluyendo funciones, clases, módulos y métodos.

Por ejemplo, es posible consultar la documentación de la función incorporada **open()**.

```
[12]: # =====
# Consultar la documentación de una función
# =====
```

```
# Mostramos la ayuda de la función open().  
help(open)
```

Help on function open in module io:

```
open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None,  
closefd=True, opener=None)
```

Open file and return a stream. Raise OSError upon failure.

file is either a text or byte string giving the name (and the path if the file isn't in the current working directory) of the file to be opened or an integer file descriptor of the file to be wrapped. (If a file descriptor is given, it is closed when the returned I/O object is closed, unless closefd is set to False.)

mode is an optional string that specifies the mode in which the file is opened. It defaults to 'r' which means open for reading in text mode. Other common values are 'w' for writing (truncating the file if it already exists), 'x' for creating and writing to a new file, and 'a' for appending (which on some Unix systems, means that all writes append to the end of the file regardless of the current seek position). In text mode, if encoding is not specified the encoding used is platform dependent: locale.getencoding() is called to get the current locale encoding.

(For reading and writing raw bytes use binary mode and leave encoding unspecified.) The available modes are:

=====	
Character Meaning	

'r'	open for reading (default)
'w'	open for writing, truncating the file first
'x'	create a new file and open it for writing
'a'	open for writing, appending to the end of the file if it exists
'b'	binary mode
't'	text mode (default)
'+'	open a disk file for updating (reading and writing)
=====	

The default mode is 'rt' (open for reading text). For binary random access, the mode 'w+b' opens and truncates the file to 0 bytes, while 'r+b' opens the file without truncation. The 'x' mode implies 'w' and raises an `FileExistsError` if the file already exists.

Python distinguishes between files opened in binary and text modes, even when the underlying operating system doesn't. Files opened in binary mode (appending 'b' to the mode argument) return contents as

bytes objects without any decoding. In text mode (the default, or when 't' is appended to the mode argument), the contents of the file are returned as strings, the bytes having been first decoded using a platform-dependent encoding or using the specified encoding if given.

buffering is an optional integer used to set the buffering policy. Pass 0 to switch buffering off (only allowed in binary mode), 1 to select line buffering (only usable in text mode), and an integer > 1 to indicate the size of a fixed-size chunk buffer. When no buffering argument is given, the default buffering policy works as follows:

- * Binary files are buffered in fixed-size chunks; the size of the buffer is chosen using a heuristic trying to determine the underlying device's "block size" and falling back on `io.DEFAULT_BUFFER_SIZE`.
On many systems, the buffer will typically be 4096 or 8192 bytes long.
- * "Interactive" text files (files for which `isatty()` returns True) use line buffering. Other text files use the policy described above for binary files.

encoding is the name of the encoding used to decode or encode the file. This should only be used in text mode. The default encoding is platform dependent, but any encoding supported by Python can be passed. See the codecs module for the list of supported encodings.

errors is an optional string that specifies how encoding errors are to be handled---this argument should not be used in binary mode. Pass 'strict' to raise a ValueError exception if there is an encoding error (the default of None has the same effect), or pass 'ignore' to ignore errors. (Note that ignoring encoding errors can lead to data loss.) See the documentation for `codecs.register` or run `'help(codecs.Codec)'` for a list of the permitted encoding error strings.

newline controls how universal newlines works (it only applies to text mode). It can be None, '', '\n', '\r', and '\r\n'. It works as follows:

- * On input, if newline is None, universal newlines mode is enabled. Lines in the input can end in '\n', '\r', or '\r\n', and these are translated into '\n' before being returned to the caller. If it is '', universal newline mode is enabled, but line endings are returned to the caller untranslated. If it has any of the other legal values, input lines are only terminated by the given string, and the line ending is returned to the caller untranslated.
- * On output, if newline is None, any '\n' characters written are translated to the system default line separator, `os.linesep`. If newline is '' or '\n', no translation takes place. If newline is any

of the other legal values, any '\n' characters written are translated to the given string.

If `closefd` is `False`, the underlying file descriptor will be kept open when the file is closed. This does not work when a file name is given and must be `True` in that case.

A custom opener can be used by passing a callable as `*opener*`. The underlying file descriptor for the file object is then obtained by calling `*opener*` with `(*file*, *flags*)`. `*opener*` must return an open file descriptor (passing `os.open` as `*opener*` results in functionality similar to passing `None`).

`open()` returns a file object whose type depends on the mode, and through which the standard file operations such as reading and writing are performed. When `open()` is used to open a file in a text mode ('w', 'r', 'wt', 'rt', etc.), it returns a `TextIOWrapper`. When used to open a file in a binary mode, the returned class varies: in read binary mode, it returns a `BufferedReader`; in write binary and append binary modes, it returns a `BufferedWriter`, and in read/write mode, it returns a `BufferedRandom`.

It is also possible to use a string or bytearray as a file for both reading and writing. For strings `StringIO` can be used like a file opened in a text mode, and for bytes a `BytesIO` can be used like a file opened in a binary mode.

5.0.2 Crear un docstring

No es necesario modificar manualmente el atributo `__doc__`.

Para documentar una función basta con escribir una cadena de texto entre **comillas triples** (`"""`) inmediatamente después de la definición de la función y antes del código ejecutable.

```
[13]: # =====
# Ejemplo de un docstring
# =====

# Definimos una función.
def funcion_de_prueba():
    """
    Esta es la documentación de la función de prueba.
    """

    # La función no realiza ninguna acción.
    pass
```

```
# Mostramos la documentación utilizando help().
help(funcion_de_prueba)
```

Help on function funcion_de_prueba in module __main__:

```
funcion_de_prueba()
    Esta es la documentación de la función de prueba.
```

5.0.3 Acceder al atributo `__doc__`

Si únicamente se desea obtener el texto del docstring, puede consultarse directamente mediante el atributo especial `__doc__`.

```
[14]: # =====
# Mostrar únicamente el docstring
# =====

# Imprimimos el contenido del atributo __doc__.
print(funcion_de_prueba.__doc__)
```

Esta es la documentación de la función de prueba.

5.0.4 Diferencia entre `help()` y `__doc__`

Aunque ambos permiten acceder a la documentación, presentan diferencias importantes:

- `help()` muestra información adicional sobre el objeto, como su firma, descripción y otros detalles útiles.
- `__doc__` devuelve únicamente el texto del docstring definido por el programador.

5.0.5 Documentación de módulos

La función `help()` también puede utilizarse para consultar la documentación completa de un módulo de Python.

En el siguiente ejemplo se visualizará la documentación del módulo `sys`.

```
[15]: # =====
# Consultar la documentación de un módulo
# =====

# Importamos el módulo sys.
import sys

# Mostramos su documentación.
help(sys)
```

Help on built-in module sys:

NAME

sys

MODULE REFERENCE

<https://docs.python.org/3.11/library/sys.html>

The following documentation is automatically generated from the Python source files. It may be incomplete, incorrect or include features that are considered implementation detail and may vary between Python implementations. When in doubt, consult the module reference at the location listed above.

DESCRIPTION

This module provides access to some objects used or maintained by the interpreter and to functions that interact strongly with the interpreter.

Dynamic objects:

argv -- command line arguments; argv[0] is the script pathname if known
path -- module search path; path[0] is the script directory, else ''
modules -- dictionary of loaded modules

displayhook -- called to show results in an interactive session
excepthook -- called to handle any uncaught exception other than SystemExit
To customize printing in an interactive session or to install a custom top-level exception handler, assign other functions to replace these.

stdin -- standard input file object; used by input()
stdout -- standard output file object; used by print()
stderr -- standard error object; used for error messages
By assigning other file objects (or objects that behave like files) to these, it is possible to redirect all of the interpreter's I/O.

last_type -- type of last uncaught exception
last_value -- value of last uncaught exception
last_traceback -- traceback of last uncaught exception
These three are only available in an interactive session after a traceback has been printed.

Static objects:

builtin_module_names -- tuple of module names built into this interpreter
copyright -- copyright notice pertaining to this interpreter
exec_prefix -- prefix used to find the machine-specific Python library
executable -- absolute path of the executable binary of the Python interpreter

float_info -- a named tuple with information about the float implementation.
 float_repr_style -- string indicating the style of repr() output for floats
 hash_info -- a named tuple with information about the hash algorithm.
 hexversion -- version information encoded as a single integer
 implementation -- Python implementation information.
 int_info -- a named tuple with information about the int implementation.
 maxsize -- the largest supported length of containers.
 maxunicode -- the value of the largest Unicode code point
 platform -- platform identifier
 prefix -- prefix used to find the Python library
 thread_info -- a named tuple with information about the thread
 implementation.
 version -- the version of this interpreter as a string
 version_info -- version information as a named tuple
 dllhandle -- [Windows only] integer handle of the Python DLL
 winver -- [Windows only] version number of the Python DLL
 _enablelegacywindowsfsencoding -- [Windows only]
 __stdin__ -- the original stdin; don't touch!
 __stdout__ -- the original stdout; don't touch!
 __stderr__ -- the original stderr; don't touch!
 __displayhook__ -- the original displayhook; don't touch!
 __excepthook__ -- the original excepthook; don't touch!

Functions:

displayhook() -- print an object to the screen, and save it in builtins._
 excepthook() -- print an exception and its traceback to sys.stderr
 exception() -- return the current thread's active exception
 exc_info() -- return information about the current thread's active exception
 exit() -- exit the interpreter by raising SystemExit
 getdlopenflags() -- returns flags to be used for dlopen() calls
 getprofile() -- get the global profiling function
 getrefcount() -- return the reference count for an object (plus one :-)
 getrecursionlimit() -- return the max recursion depth for the interpreter
 getsizeof() -- return the size of an object in bytes
 gettrace() -- get the global debug tracing function
 setdlopenflags() -- set the flags to be used for dlopen() calls
 setprofile() -- set the global profiling function
 setrecursionlimit() -- set the max recursion depth for the interpreter
 settrace() -- set the global debug tracing function

FUNCTIONS

```
__breakpointhook__ = breakpointhook(...)
    breakpointhook(*args, **kws)
```

This hook function is called by built-in breakpoint().

```
__displayhook__ = displayhook(object, /)
```

Print an object to sys.stdout and also save it in builtins._

```
__excepthook__ = excepthook(exctype, value, traceback, /)
    Handle an exception by displaying it with a traceback on sys.stderr.
```

```
__unraisablehook__ = unraisablehook(unraisable, /)
    Handle an unraisable exception.
```

The unraisable argument has the following attributes:

- * exc_type: Exception type.
- * exc_value: Exception value, can be None.
- * exc_traceback: Exception traceback, can be None.
- * err_msg: Error message, can be None.
- * object: Object causing the exception, can be None.

```
addaudithook(hook)
    Adds a new audit hook callback.
```

```
audit(...)
    audit(event, *args)
```

Passes the event to any audit hooks that are attached.

```
call_tracing(func, args, /)
    Call func(*args), while tracing is enabled.
```

The tracing state is saved, and restored afterwards. This is intended to be called from a debugger from a checkpoint, to recursively debug some other code.

```
exc_info()
    Return current exception information: (type, value, traceback).
```

Return information about the most recent exception caught by an except clause in the current stack frame or in an older stack frame.

```
exception()
    Return the current exception.
```

Return the most recent exception caught by an except clause in the current stack frame or in an older stack frame, or None if no such exception exists.

```
exit(status=None, /)
    Exit the interpreter by raising SystemExit(status).
```

If the status is omitted or None, it defaults to zero (i.e., success).

If the status is an integer, it will be used as the system exit status. If it is another kind of object, it will be printed and the system exit status will be one (i.e., failure).

`get_asyncgen_hooks()`

Return the installed asynchronous generators hooks.

This returns a namedtuple of the form (firstiter, finalizer).

`get_coroutine_origin_tracking_depth()`

Check status of origin tracking for coroutine objects in this thread.

`get_int_max_str_digits()`

Return the maximum string digits limit for non-binary int<->str conversions.

`getallocatedblocks()`

Return the number of memory blocks currently allocated.

`getdefaultencoding()`

Return the current default encoding used by the Unicode implementation.

`getfilesystemencodeerrors()`

Return the error mode used Unicode to OS filename conversion.

`getfilesystemencoding()`

Return the encoding used to convert Unicode filenames to OS filenames.

`getprofile()`

Return the profiling function set with `sys.setprofile`.

See the profiler chapter in the library manual.

`getrecursionlimit()`

Return the current value of the recursion limit.

The recursion limit is the maximum depth of the Python interpreter stack. This limit prevents infinite recursion from causing an overflow of the C stack and crashing Python.

`getrefcount(object, /)`

Return the reference count of object.

The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to `getrefcount()`.

`getsizeof(...)`

`getsizeof(object [, default]) -> int`

Return the size of object in bytes.

`getswitchinterval()`

Return the current thread switch interval; see `sys.setswitchinterval()`.

`gettrace()`

Return the global debug tracing function set with `sys.settrace`.

See the debugger chapter in the library manual.

`getwindowsversion()`

Return info about the running version of Windows as a named tuple.

The members are named: `major`, `minor`, `build`, `platform`, `service_pack`, `service_pack_major`, `service_pack_minor`, `suite_mask`, `product_type` and `platform_version`. For backward compatibility, only the first 5 items are available by indexing. All elements are numbers, except `service_pack` and `platform_type` which are strings, and `platform_version` which is a 3-tuple. Platform is always 2. Product_type may be 1 for a workstation, 2 for a domain controller, 3 for a server. Platform_version is a 3-tuple containing a version number that is intended for identifying the OS rather than feature detection.

`intern(string, /)`

``Intern'' the given string.

This enters the string in the (global) table of interned strings whose purpose is to speed up dictionary lookups. Return the string itself or the previously interned string object with the same value.

`is_finalizing()`

Return True if Python is exiting.

`set_asyncgen_hooks(...)`

`set_asyncgen_hooks([firstiter] [, finalizer])`

Set a finalizer for async generators objects.

`set_coroutine_origin_tracking_depth(depth)`

Enable or disable origin tracking for coroutine objects in this thread.

Coroutine objects will track 'depth' frames of traceback information about where they came from, available in their `cr_origin` attribute.

Set a depth of 0 to disable.

`set_int_max_str_digits(maxdigits)`
 Set the maximum string digits limit for non-binary int<->str conversions.

`setprofile(...)`
`setprofile(function)`

Set the profiling function. It will be called on each function call and return. See the profiler chapter in the library manual.

`setrecursionlimit(limit, /)`
 Set the maximum depth of the Python interpreter stack to n.

This limit prevents infinite recursion from causing an overflow of the C stack and crashing Python. The highest possible limit is platform-dependent.

`setswitchinterval(interval, /)`
 Set the ideal thread switching delay inside the Python interpreter.

The actual frequency of switching threads can be lower if the interpreter executes long sequences of uninterruptible code (this is implementation-specific and workload-dependent).

The parameter must represent the desired switching delay in seconds. A typical value is 0.005 (5 milliseconds).

`settrace(...)`
`settrace(function)`

Set the global debug tracing function. It will be called on each function call. See the debugger chapter in the library manual.

`unraisablehook(unraisable, /)`
 Handle an unraisable exception.

The unraisable argument has the following attributes:

- * `exc_type`: Exception type.
- * `exc_value`: Exception value, can be None.
- * `exc_traceback`: Exception traceback, can be None.
- * `err_msg`: Error message, can be None.
- * `object`: Object causing the exception, can be None.

DATA

```
__stderr__ = <_io.TextIOWrapper name='<stderr>' mode='w' encoding='cp1...
__stdin__  = <_io.TextIOWrapper name='<stdin>' mode='r' encoding='cp125...
__stdout__ = <_io.TextIOWrapper name='<stdout>' mode='w' encoding='cp1...
```

```

api_version = 1013
argv = [r'C:\Users\Kevin\anaconda3\envs\machine2026\Lib\site-packages\...
base_exec_prefix = r'C:\Users\Kevin\anaconda3\envs\machine2026'
base_prefix = r'C:\Users\Kevin\anaconda3\envs\machine2026'
builtin_module_names = ('_abc', '_ast', '_bisect', '_blake2', '_codecs...
byteorder = 'little'
copyright = 'Copyright (c) 2001-2023 Python Software Foundati...ematis...
displayhook = <ipykernel.displayhook.ZMQShellDisplayHook object>
dllhandle = 140714456055808
dont_write_bytecode = False
exec_prefix = r'C:\Users\Kevin\anaconda3\envs\machine2026'
executable = r'C:\Users\Kevin\anaconda3\envs\machine2026\python.exe'
flags = sys.flags(debug=0, inspect=0, interactive=0, opt...coding=0, s...
float_info = sys.float_info(max=1.7976931348623157e+308, max_...epsilo...
float_repr_style = 'short'
hash_info = sys.hash_info(width=64, modulus=2305843009213693...iphash1...
hexversion = 51056624
implementation = namespace(name='cpython', cache_tag='cpython-311...as...
int_info = sys.int_info(bits_per_digit=30, sizeof_digit=4, ..._str_dig...
maxsize = 9223372036854775807
maxunicode = 1114111
meta_path = [<_distutils_hack.DistutilsMetaFinder object>, <class '_fr...
modules = {'IPython': <module 'IPython' from 'C:\\Users\\Kevin\\anacon...
orig_argv = [r'C:/Users/Kevin/anaconda3/envs/machine2026/python.exe', ...
path = [r'C:\Users\Kevin\anaconda3\envs\machine2026\python311.zip', r'...
path_hooks = [<class 'zipimport.zipimporter'>, <function FileFinder.pa...
path_importer_cache = {r'C:\Users\Kevin\Desktop\machine\practicas': Fi...
platform = 'win32'
platlibdir = 'DLLs'
prefix = r'C:\Users\Kevin\anaconda3\envs\machine2026'
ps1 = 'In : '
ps2 = '...: '
ps3 = 'Out: '
pycache_prefix = None
stderr = <ipykernel.iostream.OutStream object>
stdin = <_io.TextIOWrapper name='<stdin>' mode='r' encoding='cp1252'>
stdlib_module_names = frozenset({'__future__', '_abc', '_aix_support',...
stdout = <ipykernel.iostream.OutStream object>
thread_info = sys.thread_info(name='nt', lock=None, version=None)
version = '3.11.15 | packaged by Anaconda, Inc. | (main, Jun 11 2026, ...
version_info = sys.version_info(major=3, minor=11, micro=15, releasele...
warnoptions = []
winver = '3.11'

```

FILE

(built-in)

5.0.6 Estructura recomendada de un docstring

Cuando un docstring ocupa varias líneas, normalmente sigue la siguiente estructura:

- Un resumen breve en la primera línea.
- Una línea en blanco.
- Una descripción más detallada.
- Otra línea en blanco antes del código de la función.

Este formato facilita la lectura tanto para los desarrolladores como para las herramientas de documentación automática.

```
[16]: # =====  
# Ejemplo de un docstring multilínea  
# =====  
  
# Definimos una función con documentación extensa.  
def funcion_de_prueba2():  
    """  
    Esta es la línea para el resumen.  
  
    Este es el párrafo donde se puede escribir una descripción  
    más detallada de la función. Observa que el resumen y la  
    descripción van separados por una línea en blanco.  
  
    También existe otra línea en blanco antes de comenzar  
    el código de la función.  
    """  
  
    # La función no realiza ninguna acción.  
    pass  
  
# Mostramos la documentación generada.  
help(funcion_de_prueba2)
```

Help on function funcion_de_prueba2 in module __main__:

```
funcion_de_prueba2()  
    Esta es la línea para el resumen.  
  
    Este es el párrafo donde se puede escribir una descripción  
    más detallada de la función. Observa que el resumen y la  
    descripción van separados por una línea en blanco.  
  
    También existe otra línea en blanco antes de comenzar  
    el código de la función.
```

5.0.7 Conclusión

Los **docstrings** constituyen una buena práctica de programación, ya que permiten documentar el propósito y funcionamiento del código directamente dentro del programa.

Además de mejorar la legibilidad, facilitan el mantenimiento del software y permiten que herramientas como **help()** o los generadores automáticos de documentación obtengan información detallada sobre módulos, funciones y clases.

5.0.8 Comentarios y Docstrings

Aunque tanto los comentarios como los docstrings sirven para documentar el código, cada uno tiene un propósito diferente.

- Los **comentarios** (#) suelen utilizarse para explicar instrucciones individuales, expresiones o pequeños bloques de código.
- Los **docstrings** están pensados para documentar elementos de mayor nivel, como módulos, funciones, clases y métodos.
- Los docstrings pueden consultarse durante la ejecución del programa mediante la función **help()** o el atributo **__doc__**, mientras que los comentarios únicamente sirven como ayuda para el programador y son ignorados por Python durante la ejecución.

5.1 2.1 Sphinx

Sphinx es una herramienta especializada en la generación automática de documentación para proyectos desarrollados en Python.

Su principal ventaja es que puede generar documentación profesional a partir de los docstrings escritos dentro del código fuente.

Entre sus características más importantes se encuentran:

- Generación automática de documentación.
- Ideal para proyectos de mediana y gran escala.
- Compatible con múltiples formatos de salida, entre ellos:
 - HTML
 - PDF (mediante LaTeX)
 - ePub
 - Manuales en línea
- Integración con plataformas de documentación y repositorios de código.

5.1.1 ¿Por qué utilizar Sphinx?

Cuando un proyecto crece y contiene decenas o cientos de módulos, mantener una documentación manual resulta complicado.

Sphinx permite generar automáticamente páginas de documentación utilizando la información almacenada en los **docstrings**, evitando duplicar esfuerzos y manteniendo la documentación sincronizada con el código.

5.2 2.2 Estilo

Es recomendable que todos los docstrings de un proyecto mantengan un formato uniforme.

Cuando varias personas trabajan sobre el mismo código, seguir un mismo estilo facilita la lectura, el mantenimiento y la generación automática de documentación.

Por esta razón, los equipos de desarrollo suelen definir una guía de estilo antes de comenzar un proyecto.

5.2.1 Google Python Style Guide

Una de las guías de estilo más utilizadas en proyectos profesionales es la **Google Python Style Guide**.

Esta guía establece recomendaciones sobre cómo documentar correctamente los diferentes componentes de un programa, entre ellos:

- Módulos.
- Funciones y métodos.
- Clases.
- Parámetros.
- Valores de retorno.
- Excepciones generadas.

Seguir una guía de estilo permite que la documentación sea consistente y fácil de comprender para cualquier desarrollador que participe en el proyecto.

5.2.2 Conclusión

Una buena documentación es tan importante como un buen código.

El uso adecuado de **docstrings**, junto con herramientas como **Sphinx** y una guía de estilo consistente, facilita el mantenimiento de los programas, mejora la colaboración entre desarrolladores y permite generar documentación profesional de forma automática.

6 2.3 Ejercicios

A continuación se presentan algunos ejercicios para practicar la lectura y escritura de archivos de texto, así como el procesamiento de información almacenada en ficheros utilizando Python.

6.1 Ejercicio 1

Enunciado

Escribe una función que reciba la ruta de un fichero de texto y una cadena de caracteres, y determine si dicha cadena aparece dentro del archivo.

```
[17]: # =====  
# Ejercicio 1  
# Buscar una cadena dentro de un archivo  
# =====  
  
# Importamos el módulo os para construir rutas.  
import os
```

```

# Definimos la función.
def buscar_cadena(ruta, cadena):

    # Abrimos el archivo en modo lectura.
    with open(ruta, "r", encoding="utf-8") as fichero:

        # Leemos todo el contenido del archivo.
        contenido = fichero.read()

        # Verificamos si la cadena existe.
        return cadena in contenido

# Ruta del archivo de prueba.
ruta = os.path.join("res", "multiple_lines.txt")

# Probamos la función.
print(buscar_cadena(ruta, "Python"))
print(buscar_cadena(ruta, "Java"))

```

False

False

6.2 Ejercicio 2

Enunciado

Escribe una función que reciba una lista, una ruta de destino y un número **n**. La función debe crear un archivo que contenga únicamente los primeros **n** elementos de la lista, controlando adecuadamente los valores fuera de rango.

```

[18]: # =====
# Ejercicio 2
# Guardar los primeros n elementos
# =====

# Definimos la función.
def guardar_elementos(lista, ruta, n):

    # Verificamos que n sea válido.
    if n < 0 or n > len(lista):
        print("Error: valor de n fuera de rango.")
        return

    # Creamos el archivo.
    with open(ruta, "w", encoding="utf-8") as fichero:

```

```

        # Escribimos únicamente los primeros n elementos.
        for elemento in lista[:n]:
            fichero.write(str(elemento) + "\n")

# Lista de ejemplo.
datos = ["Ana", "Luis", "Pedro", "María", "José"]

# Ruta del archivo.
ruta = os.path.join("res", "primeros_elementos.txt")

# Ejecutamos la función.
guardar_elementos(datos, ruta, 3)

print("Archivo generado correctamente.")

```

Archivo generado correctamente.

6.3 Ejercicio 3

Enunciado

Escribe una función que reciba la ruta de un fichero de texto y devuelva un diccionario con la frecuencia de aparición de cada palabra contenida en el archivo.

```

[20]: # =====
# Ejercicio 3
# Contar frecuencia de palabras
# =====

# Definimos la función.
def frecuencia_palabras(ruta):

    # Abrimos el archivo.
    with open(ruta, "r", encoding="utf-8") as fichero:

        # Leemos el contenido.
        texto = fichero.read()

        # Dividimos el texto en palabras.
        palabras = texto.split()

        # Diccionario donde almacenaremos las frecuencias.
        frecuencias = {}

        # Recorremos todas las palabras.
        for palabra in palabras:

```

```

    # Si la palabra ya existe, incrementamos el contador.
    if palabra in frecuencias:
        frecuencias[palabra] += 1

    # Si aparece por primera vez, iniciamos el contador.
    else:
        frecuencias[palabra] = 1

    # Devolvemos el diccionario.
    return frecuencias

# Ruta del archivo de ejemplo.
ruta = os.path.join("res", "primeros_elementos.txt")

# Mostramos el resultado.
print(frecuencia_palabras(ruta))

```

```
{'Ana': 1, 'Luis': 1, 'Pedro': 1}
```

6.3.1 Conclusión

Con estos ejercicios se aplican los principales conceptos estudiados en este capítulo, como la lectura y escritura de archivos de texto, el manejo de rutas mediante el módulo **os**, el uso de **context managers** con la sentencia **with** y el procesamiento de información utilizando estructuras de datos como listas y diccionarios.